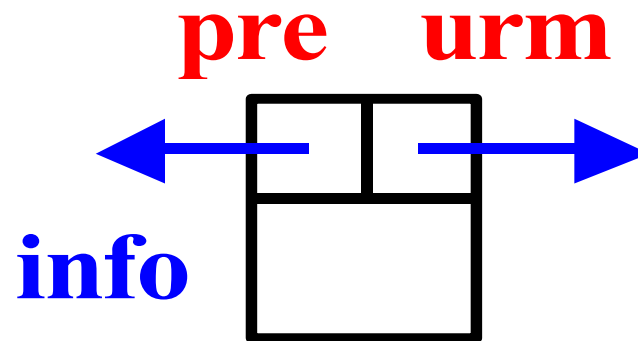


Liste dublu inlantuite

Celulele din lista au doua campuri legatura:

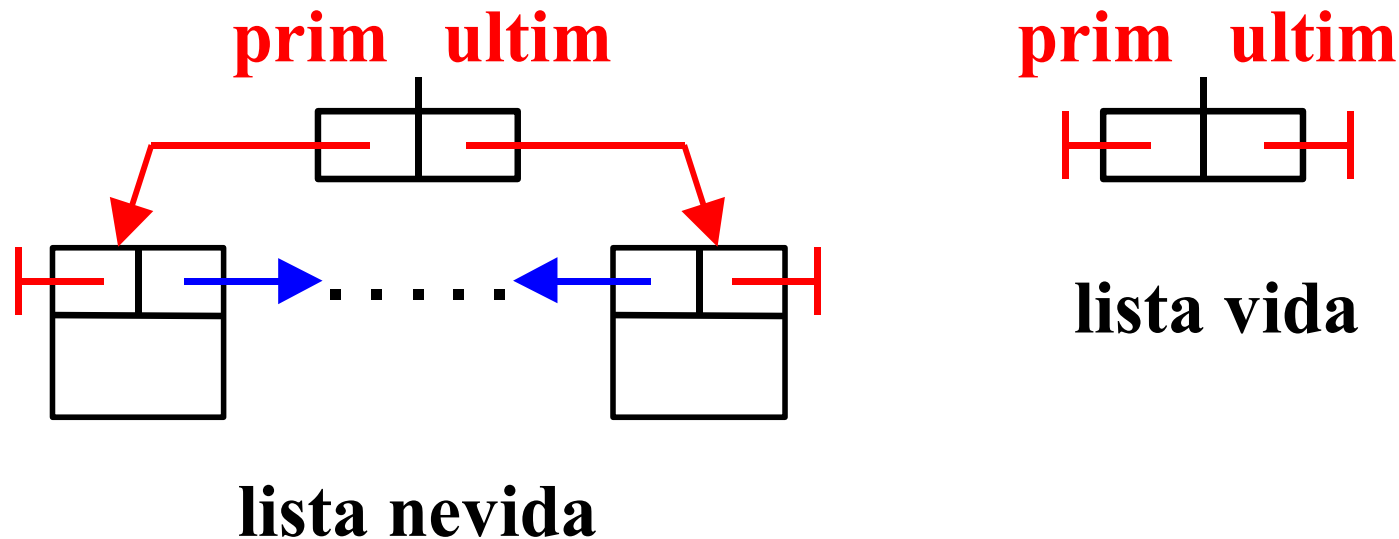
pre – adresa celulei predecesoare

urm – adresa celulei urmatoare



Pentru **parcurgerea eficienta** a listei in oricare din cele doua sensuri (**de la inceput** sau **de la sfarsit**) este necesar accesul la cele doua extremitati.

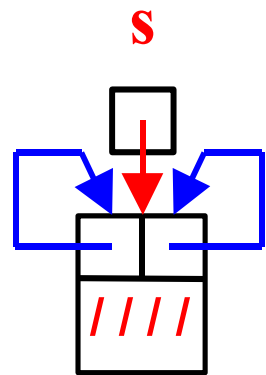
Liste dublu inlantuite deschise



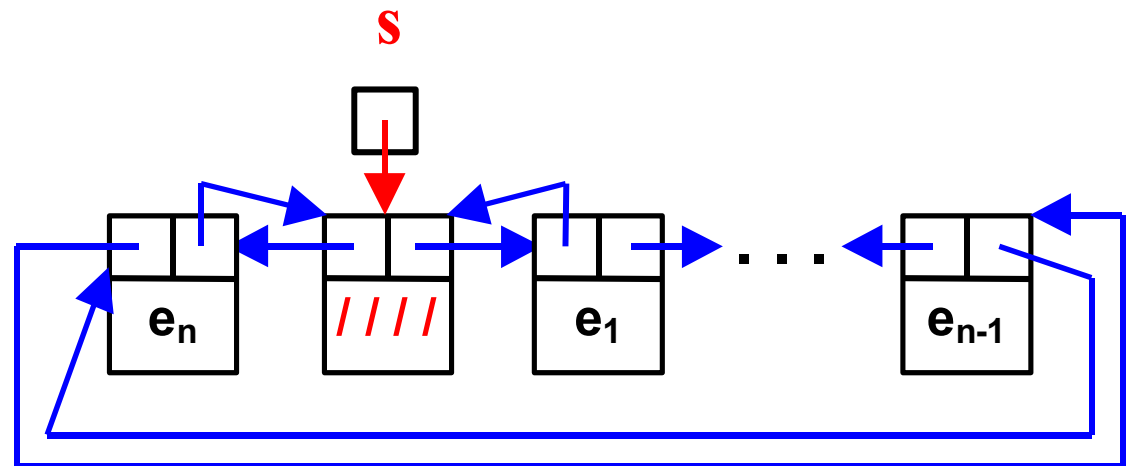
Se observa ca in aceasta abordare exista **diferente de tratare** in cazul modificarilor (inserare / eliminare) **la extremitati**, deoarece acestea efecteaza fie campul **prim**, fie campul **ultim**.

Liste dublu inlantuite circulare

Pentru o **tratare uniforma** se poate adopta solutia de organizare ca **lista circulara cu santinela**.



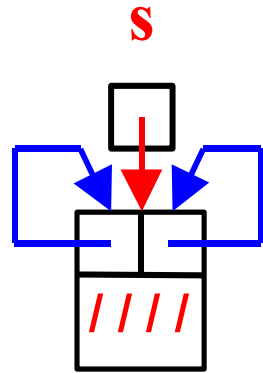
lista vida



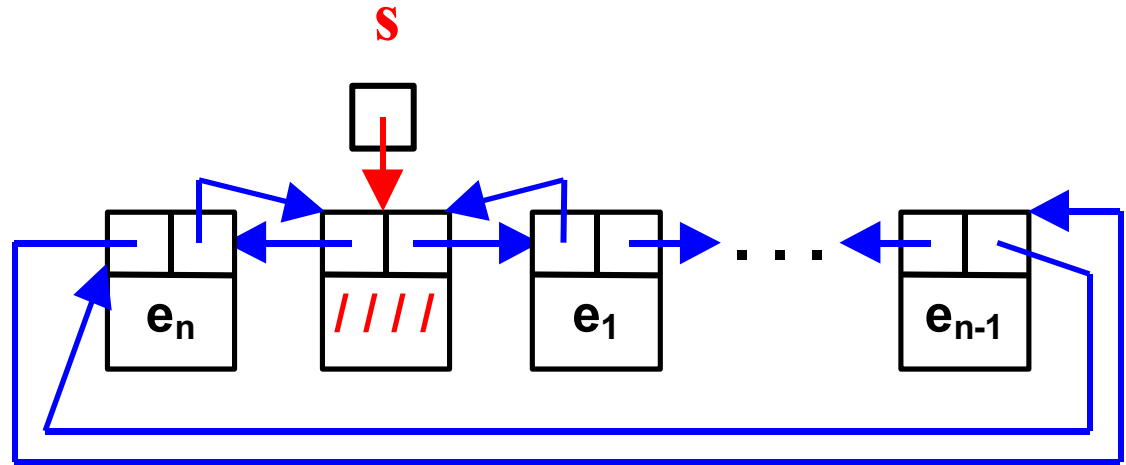
lista nevida

Campul **info** al celulei **santinela** poate fi utilizat pentru pastrarea unor informatii cu caracter general (de ex. referinta la functia de verificare a relatiei de ordine in cazul in care elementele din lista sunt ordonate).

Liste dublu inlantuite circulare



lista vida



lista nevida

```
typedef struct cel2
{ struct cel2 *pre, *urm;
  void* info;
} T Cel2, *TL2;
```

Liste dublu inlantuite circulare

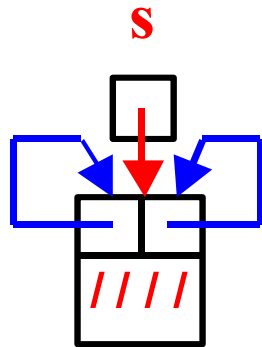
Test lista vida:

$(s \rightarrow \text{urm} == s)$ sau $(s \rightarrow \text{pre} == s)$

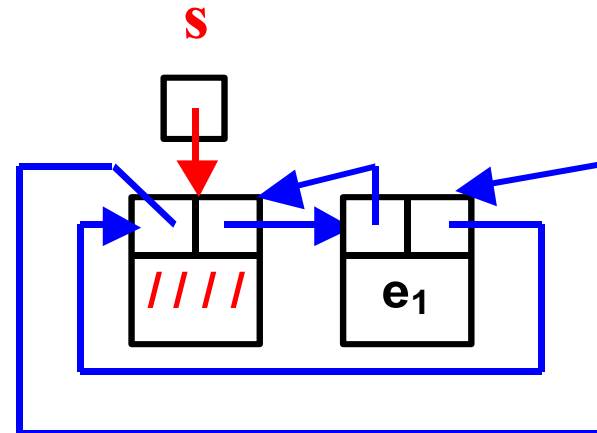
Atentie! Expresia

$(s \rightarrow \text{urm} == s \rightarrow \text{pre})$

are valoarea 1 si in cazul listei cu un singur element.



lista vida

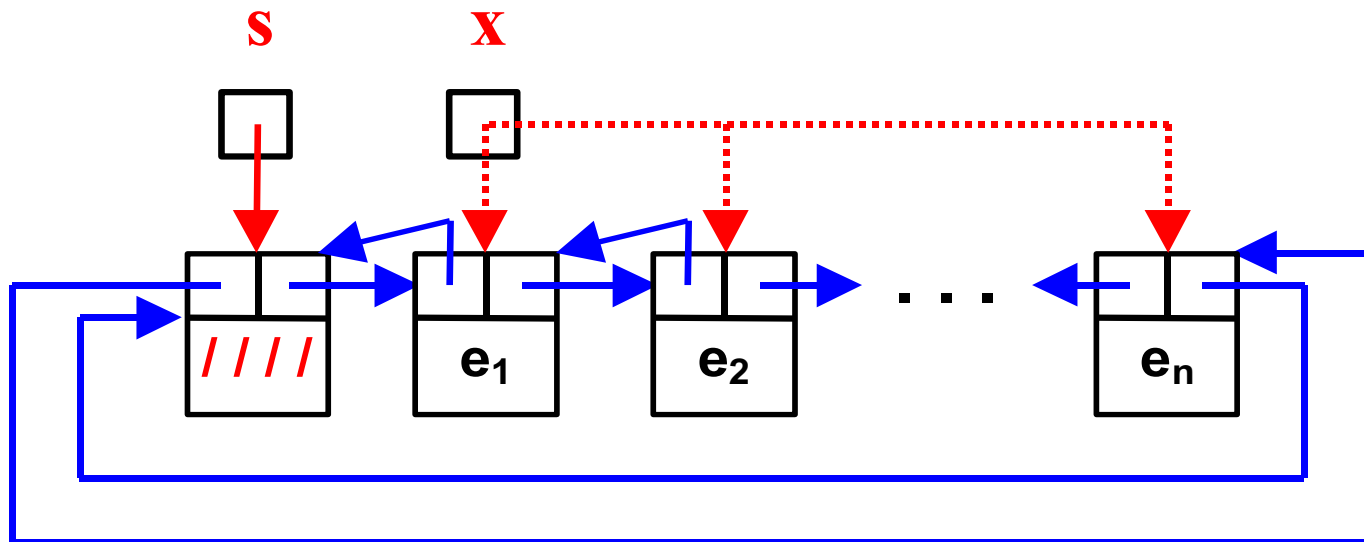


lista cu un element

Parcurgerea listei dublu inlantuite circulare

- de la inceput

```
for (x = s->urm; x != s; x = x->urm)  
{ . . . }
```



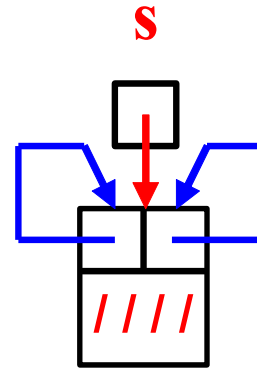
- de la sfarsit

```
for (x = s->pre; x != s; x = x->pre)  
{ . . . }
```

Initializare lista dublu inlantuita circulara

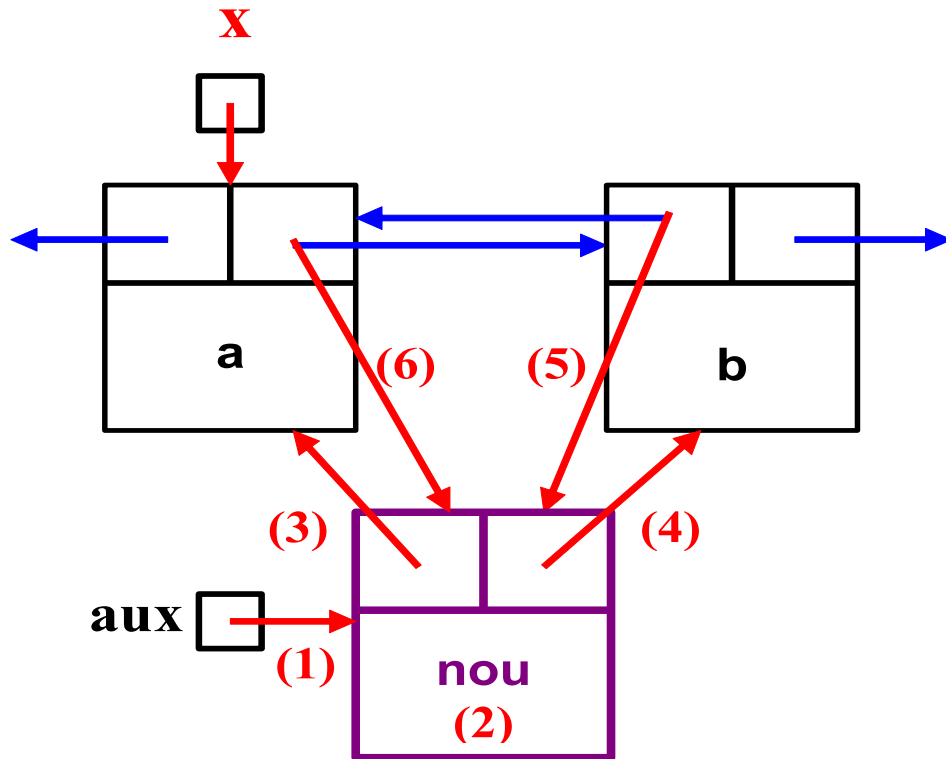
```
TL2 InitL2()
```

```
{  
    TL2 aux = (TL2)malloc(sizeof(TCel2));  
    if(aux)  
    { aux->pre = aux->urm = aux;  
      aux->info = NULL;  
    }  
    return aux;  
}
```



lista vida

Inserare dupa celula cu adresa x



Secventa de operatii:

- **alocare spatiu** pentru o noua celula (**1**)
- **actualizarea campurilor** noii celule (**2,3,4**)
- **actualizare referinte** la noua celula (**5,6**)

Inserare dupa celula cu adresa p

```
int InsDupa(TL2 a, void* x)
```

```
/* inserare dupa celula cu adresa a -> 1/0 */
```

```
{
```

```
    TL2 aux = (TL2)malloc(sizeof(TCelula2));
```

```
    if(!aux) return 0;
```

```
    aux->info = x; /*2*/
```

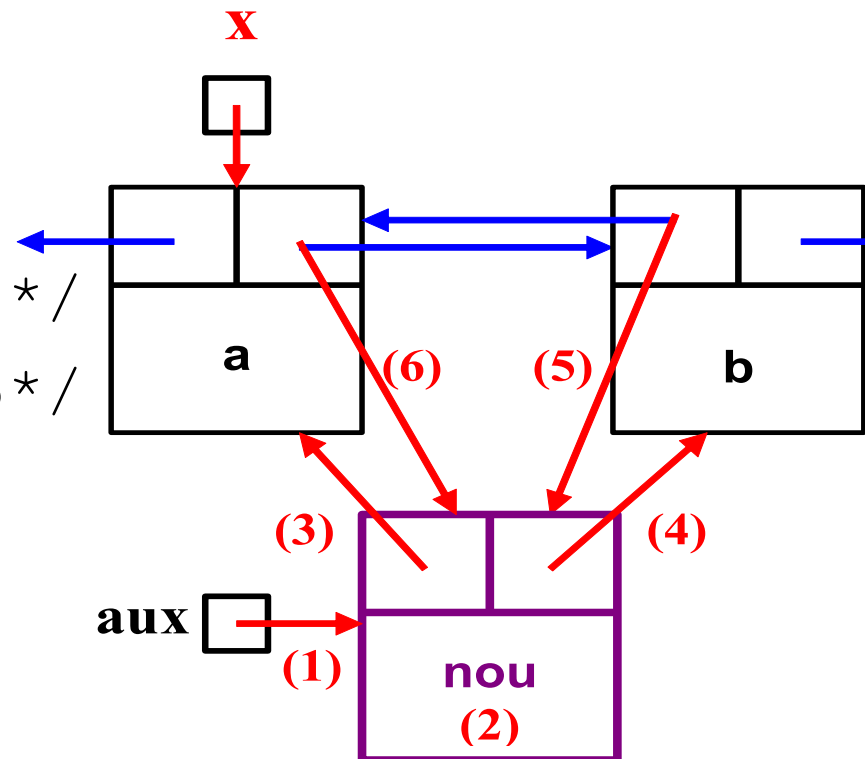
```
    aux->pre = a; /*3*/
```

```
    aux->urm = a->urm; /*4*/
```

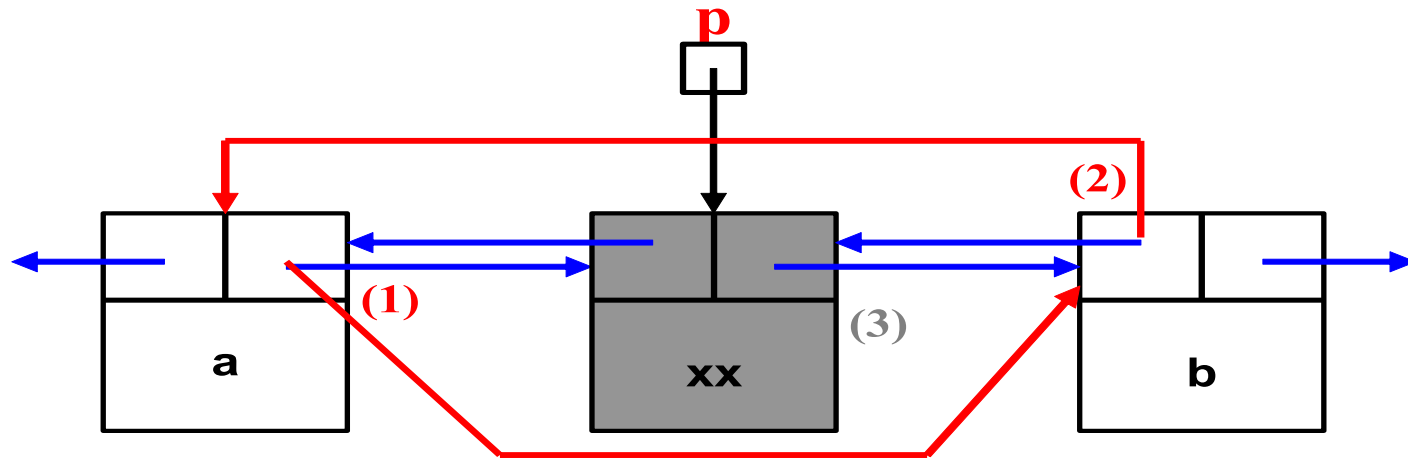
```
    a->urm->pre = aux; /*5*/
```

```
    a->urm = aux; /*6*/
```

```
}
```



Eliminarea celulei de la adresa p ($p \neq s$)



Secventa de operatii:

- actualizare legaturi in celulele vecine (1,2)
- eliberarea spatiului ocupat de celula (3)

Daca informatia din celula este necesara pentru prelucrari ulterioare, atunci ea trebuie salvata inainte de eliminarea celulei.

Eliminarea celulei de la adresa p ($p \neq s$)

```
void* ExtrL2 (TL2 p)
```

```
/* elimina celula de la adresa p
```

```
   intoarce adresa info */
```

```
{ void* r;
```

```
  if (p == p->urm) return NULL;
```

```
  r = p->info;
```

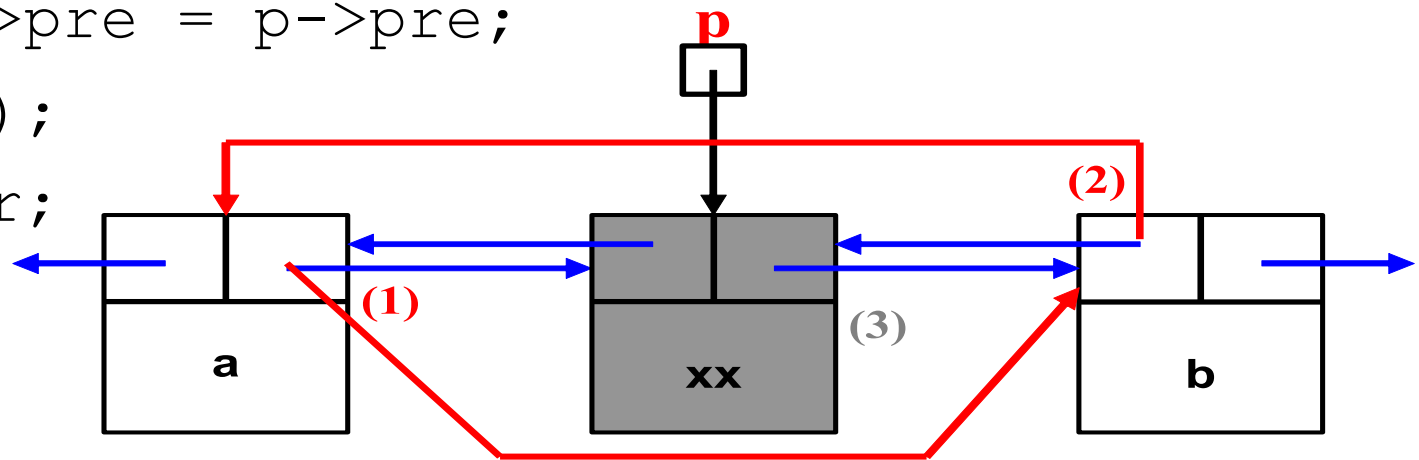
```
  p->pre->urm = p->urm;
```

```
  p->urm->pre = p->pre;
```

```
  free (p);
```

```
  return r;
```

```
}
```



Eliberare spatiul de memorie

/*- functii de eliberare spatiu; parametrul fe -
functie care elibereaza spatiul ocupat de un element */

void ResetL2(TL2 s, void (*fe)(void*))

/* transforma lista in lista vida */

```
{ TL2 p = s->urm, aux;
  while(p != s)
  {
    aux = p;
    p = p->urm;
    fe(aux->info); free(aux);
  }
  s->urm = s->pre = s;
}
```

Eliberare tot spatiul de memorie

```
void DistrL2(TL2* as, void (*fe)(void*))  
/* elibereaza tot spatiul ocupat de lista,  
inclusiv santinela */  
{  
    ResetL2(*as, fe);  
    free(*as);  
    *as = NULL;  
}
```